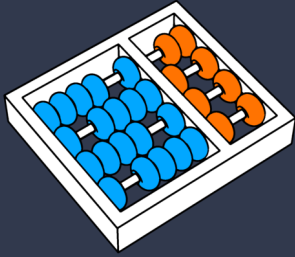




UNICAMP



Avançando com Prompts

Prof. Dr. André Gomes Regino

regino@unicamp.br

Agenda



01 0 Limite do Prompt Simples

02 Self-Consistency

03 Self-Refinement

04 Role Prompting

05 Código & Implementação

Objetivos

Prompts bem escritos ainda falham devido à natureza probabilística

Exemplo: Cálculo Matemático

Input:

"Qual a raiz quadrada de 144?"

Run 1: "12" (Correto)

Run 2: "12" (Correto)

Run 3: "11" (Alucinação/Erro)

Questão Central:

Como garantir a resposta correta sem mudar o modelo ou treinar do zero?

Nova abordagem: Orquestração de Inferência

- Paradigma Shift: De "Prompt Único" para "Fluxos de Prompting"
- Estratégia:
 - Não confiar em uma **única amostra** (N=1)
 - Usar o próprio **modelo como validador** ou crítico
 - **Explorar a variância** a favor do sistema
 - Conceito Técnico: Usar mais inferência (compute time) para ganhar precisão

Objetivos

- **Implementar** votação para aumentar a precisão em tarefas lógicas
- **Construir** loops de feedback onde o modelo critica sua própria saída
- **Utilizar** Persona Prompting para ativar domínios específicos
- **Analisar** o trade-off entre custo e qualidade da resposta

Três Estratégias

- Self-consistency
- Self-refinement
- Role prompting

Self-consistency

Intuição



Conceito

Um problema complexo pode ter múltiplos caminhos de raciocínio.



A Falha

O modelo pode "escolher" um caminho errado por probabilidade.



A Solução

Gerar **N** raciocínios independentes.

Se a maioria chega à resposta **A**, é estatisticamente mais provável que **A** esteja correta.



Analogia

Pedir ajuda a vários especialistas e votar.

Funcionamento: Fluxo de Processo



1. Amostragem (N vezes)

- **Temperature:** 0.5 a 1.0 para diversidade.
- **N ideal:** 5 a 20 execuções.
- **Técnica:** Combine com Chain-of-Thought (CoT).



2. Coleta e Parsing

- **Foco:** Extraia apenas a conclusão final.
- **JSON:** Parsing direto dos campos.
- **Texto:** Captura da última frase ou veredito.



3. Agregação

- **Majority:** Resposta mais frequente (moda).
- **Weighted:** Ponderação por logprobs.
- **Números:** Mediana para maior robustez.

Funcionamento

```
1 from openai import OpenAI
2 client = OpenAI()
3
4 prompt = "Quanto é 17 x 24? Explique o
   raciocínio."
5
6 answers = []
7
8 for _ in range(5):
9     response = client.responses.create(
10         model="gpt-5",
11         input=prompt
12     )
13     answers.append(response.output_text)
14
15 for a in answers:
16     print(a)
```

Funcionamento: Votação por Maioria

"Se 4 de 5 pessoas independentes chegam à mesma resposta, a probabilidade de estarem corretas é muito superior a confiar em apenas uma."

Prompt

(com Chain-of-Thought)

Execução 1 —▶ Raciocínio A —▶ Resposta: "408" ✓

Execução 2 —▶ Raciocínio B —▶ Resposta: "408" ✓

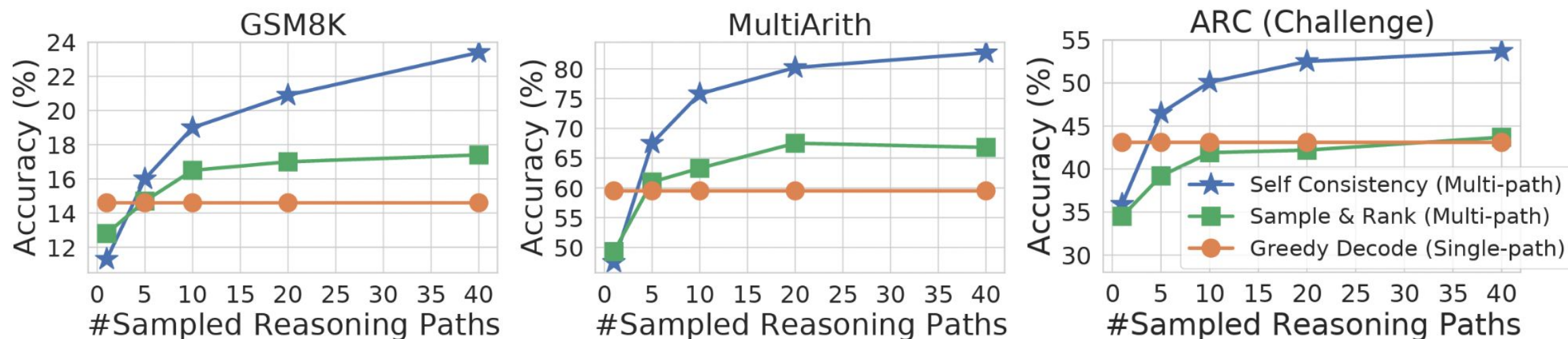
Execução 3 —▶ Raciocínio C —▶ Resposta: "312" ✗

Execução 4 —▶ Raciocínio D —▶ Resposta: "408" ✓

Execução 5 —▶ Raciocínio E —▶ Resposta: "408" ✓

Votação Majoritária: "408" (4/5) ✓

Resultados de Aplicação de Self-consistency



WEI, Jason et al. Chain-of-thought prompting elicits reasoning in large language models. **Advances in neural information processing systems**, v. 35, p. 24824-24837, 2022.

Achados do Artigo



Matemática (GSM8K)

+17,9 pp

Modelo: 540B PaLM



Raciocínio Lógico

+6,4 pp

Benchmark: StrategyQA



QA Multi-passos

+8-12 pp

Dependendo do modelo

WANG, Xuezhi et al. Self-consistency improves chain of thought reasoning in language models. [arXiv preprint arXiv:2203.11171](#), 2022.

Caso Real

Casos de Uso em Sistemas Reais

✓ Diagnóstico Automatizado

Classificação médica, jurídica ou financeira onde erros têm alto custo.

✓ QA sobre Documentos Críticos

Contratos, laudos, regulações.

✓ Extração de Dados Estruturados

Quando inconsistência de formato é inaceitável.

Quando NÃO Usar

✗ Tarefas Criativas

Respostas diversas são desejadas, não problemáticas.

✗ Sistemas com Latência Crítica

$N \text{ chamadas} = N \times \text{o tempo de resposta.}$

✗ Alto Volume + Custo Restrito

Custo escala linearmente com N .

Self-refinement

a primeira resposta nem sempre é a melhor

Intuição

- O modelo pode revisar a própria resposta para atingir maior precisão.



Funcionamento: Votação por Maioria

1. GERAÇÃO

Prompt inicial:

Prompt normal com a tarefa. Sem critérios de qualidade ainda.

Resultado:

`{resposta_inicial}`

2. CRÍTICA

Avaliação Baseada em Critérios:

- **Clareza:** Compreensível?
- **Compleitude:** Pontos essenciais?
- **Concisão:** Sem redundância?

```
JSON Response Format:  
{ "clareza": { "nota": N, "sugestao":  
  "..."}, ... }
```

3. REFINAMENTO

Incorporação de Feedback:

Reescreve a resposta incorporando o `{feedback_estruturado}`.

💡 Parada:

Nota mínima atingida ou máx de 2–3 iterações.

O prompt de revisão é o componente mais crítico para o sucesso do refinamento iterativo.

Exemplo Prático

```
1 prompt_1 = "Explique redes neurais em 5  
linhas."  
2  
3 response_1 = client.responses.create(  
4     model="gpt-5",  
5     input=prompt_1  
6 )  
7  
8 initial_answer = response_1.output_text  
9  
10 prompt_2 = f""  
11 Revise e melhore a seguinte resposta,  
deixando mais clara:  
12  
13 {initial_answer}  
14 ""  
15  
16 response_2 = client.responses.create(  
17     model="gpt-5",  
18     input=prompt_2  
19 )  
20  
21 print(response_2.output_text)
```

Achados do Artigo

Resultados Reportados: Melhoria vs. Geração Simples



Geração de Código

Melhoria de legibilidade (avaliação humana)

+13%



Escrita Criativa

Aumento em coerência e fluência do texto

+14%



Resposta a E-mails

Melhoria significativa em clareza e tom

+18%



Diálogo

Aumento da naturalidade nas interações

+11%

Caso Real

Casos de Uso em Sistemas Reais



Geração de Textos

E-mails de suporte e respostas automáticas de chatbot.



Geração de Relatórios

O modelo gera, critica a completude e reescreve o conteúdo.



Code Review

Avalia boas práticas, gera código e realiza refatoração automática.

Quando NÃO Usar



Tarefa Objetiva: Quando há resposta única (prefira self-consistency).



Latência Crítica: Chamadas sequenciais aumentam drasticamente o tempo de resposta.



Modelos Pequenos: Quando o modelo não possui capacidade de autocrítica confiável.

Role-prompting

O que você diz ao modelo que ele é muda o que ele diz que sabe

Intuição



O modelo responde diferente dependendo do “papel”

Evidência Científica

Salewski et al. (2023) - NeurIPS 2023

- **Melhora de Desempenho:** Atribuir papéis de especialistas a LLMs otimiza tarefas de domínio específico.
- **Risco de Vieses:** Papéis inadequados podem introduzir distorções sistemáticas nos resultados.



Role Prompting: Técnica de menor custo — uma única chamada com papel definido no `system prompt`.

Exemplo Prático

```
1 from openai import OpenAI
2 client = OpenAI()
3
4 pergunta = "O que é pressão alta e quais são os riscos?"
5
6 papeis = [
7     "Você é um médico cardiologista explicando para um paciente leigo.",
8     "Você é um professor universitário de fisiologia explicando para alunos de medicina.",
9     "Você é um jornalista de saúde escrevendo para uma revista popular.",
10 ]
11
12 for papel in papeis:
13     r = client.chat.completions.create(
14         model="gpt-4o",
15         messages=[
16             {"role": "system", "content": papel},
17             {"role": "user", "content": pergunta}
18         ]
19     )
20     print(f"\n--- Papel: {papel[:50]}... ---")
21     print(r.choices[0].message.content[:300])
```


Quando Usar

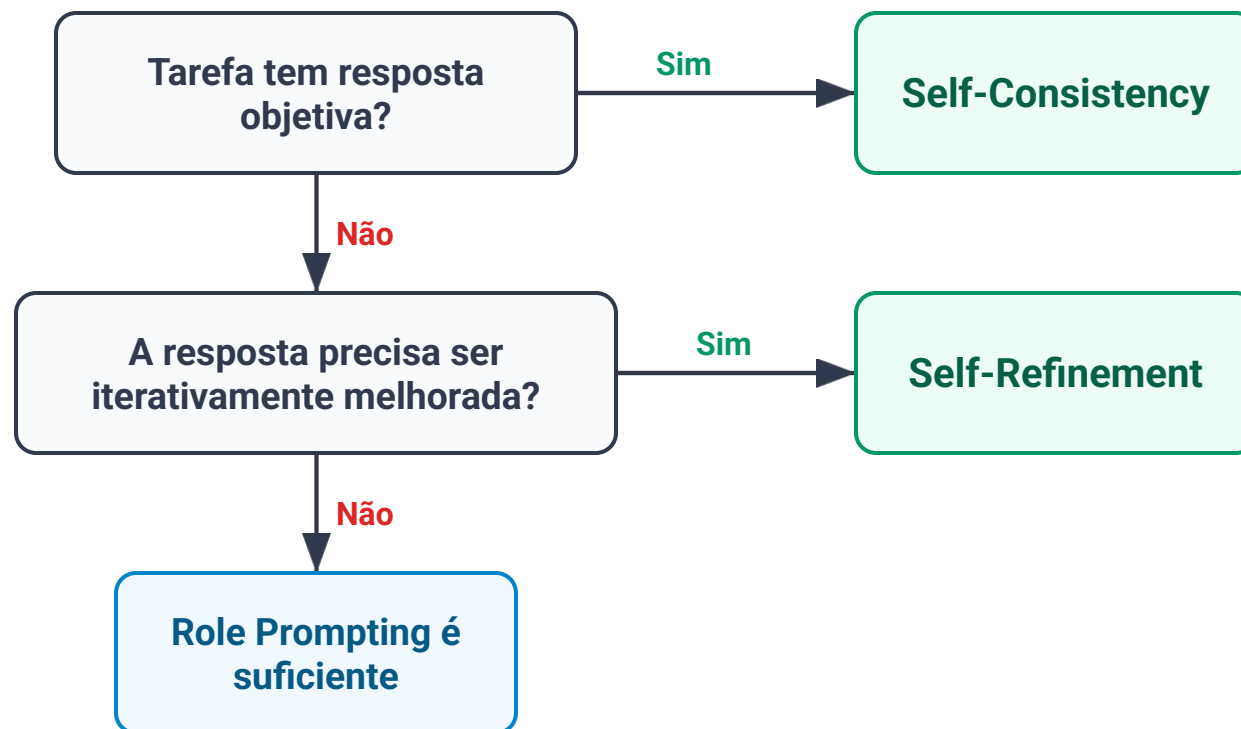
Critério	Self-consistency	Self-refinement	Role prompting
Tipo de tarefa	Raciocínio, cálculo, QA	Geração de texto, código	Qualquer tarefa com audiência específica
Resposta tem certo/errado claro	Ideal	Não	Não
Qualidade subjetiva importa	Não	Ideal	Parcial
Custo por requisição	Alto (N x custo)	Médio	Baixo
Latência tolerada	Alta	Média	Baixa
Implementação	Moderada	Moderada	Simples

Guia Rápido



Fluxo de Decisão

Use este guia para selecionar a melhor técnica de prompting avançado baseado nas características da sua tarefa.



Take-home Message



Paradigma

Um prompt avançado não é apenas texto longo, é uma **arquitetura de controle**.



Poder da Inferência

É possível melhorar a performance de modelos "burros" usando **estratégias inteligentes** de inferência.



Engenharia

O futuro é a **orquestração de múltiplos chamados (Agentes)**, não prompts únicos estáticos.